
Predicting YikYak Post Engagement from Text and Metadata

Ethan Morchy
Computer Engineering – ECE
A18538707

Abstract

YikYak is an anonymous messaging platform that allows college students to create and view discussions at UCSD. Posts and comments can be voted positively or negatively by the users, denoted as a “vote total” in the community. By leveraging Machine Learning (ML) and Natural Language Processing (NLP), previous messages can be used to predict the average vote count that a new post would receive. My approach uses regression on a normalized weekly percentile target (from $\log(1 + \text{likes})$).

The main method is a BERT pipeline: domain-adaptive pretraining (DAPT) on posts and comments, task-adaptive pretraining (TAPT) on posts, then supervised regression with frozen-then-unfrozen BERT. I compare to a TF-IDF+Ridge baseline and to MLP, transformer, and end-to-end DistilBERT models. The dataset is YikYak posts (text, timestamp, vote total) with temporal or random 70/15/15 splits. The BERT pipeline achieves Test RMSE 0.2825 and MAE 0.2340 (DAPT 3, TAPT 90, frozen 30; 0.8 non-random split) and improves over the baseline (RMSE 0.2952, MAE 0.2457). Only 0.4% of posts are truncated at max token length 64.

1 Introduction

Problem. I predict the vote total (engagement) for a new YikYak post. Engagement is heavy-tailed and time-dependent.

Motivation. Users often wonder whether a message will be well received. Predicting vote totals can help them adjust wording or timing to improve reach and positive engagement.

Understanding. Time of day and week, as well as message content and style (length, punctuation, etc.), affect votes. Regression in a normalized space is more stable than predicting raw counts.

Approach. I use a weekly-normalized percentile of $\log(1 + \text{likes})$ as the regression target. The main method is a BERT pipeline: DAPT (masked LM on posts and comments), TAPT (masked LM on posts only), then supervised regression with BERT frozen then unfrozen, and auxiliary features (time, length, hashtags, etc.) are fused with the [CLS] embedding. I compare to a TF-IDF+Ridge baseline and to MLP, transformer, and end-to-end DistilBERT models.

Results. The BERT pipeline (DAPT 3, TAPT 90, frozen 30 epochs; 0.8 non-random split) achieves Test RMSE 0.2825 and MAE 0.2340 and improves over the baseline (RMSE 0.2952, MAE 0.2457). I briefly summarize ablations and classification results.

2 Related Work

1. **Engagement and popularity prediction.** Predicting likes, retweets, or virality from social media text and metadata is well studied. Cheng et al. [1] fine-tune multilingual BERT (MBERT, XLM-R) on Twitter data to predict engagement (likes, retweets, replies), combining

tweet content with user and engagement history. Their approach placed second in the 2020 RecSys Challenge. I follow a similar pipeline on YikYak with a pretrained transformer on post text and fuse it with auxiliary features (time, length, hashtags, etc.) for regression. Unlike their work, I adapt the language model and task via domain and task-adaptive pretraining before supervised fine-tuning.

2. **Pre-trained language models.** I use BERT [2] as the text encoder. Devlin et al. show that bidirectional pretraining (masked LM and next-sentence prediction) yields representations that transfer well to downstream tasks with minimal task-specific architecture. I keep this knowledge and adapt it to YikYak with continued pretraining, then add a regression head and auxiliary tower. I use differential learning rates so the pretrained layers are updated gently while the new head is trained at a higher rate.
3. **Domain- and task-adaptive pretraining.** Gururangan et al. [3] show that continuing pretraining on in-domain text (DAPT) and then on task-specific unlabeled data (TAPT) improves performance over using a fixed pretrained model. I adopt this pipeline by running DAPT on the corpus of YikYak posts and comments (domain adaptation), then TAPT on posts only with labels stripped (task adaptation), and finally supervised regression with a frozen and optional unfrozen BERT. This matches the setting where there is substantial in-domain text but limited labeled data for engagement prediction.

3 Method

Data and target (regression). Inputs: post text and `created_at`. Output: percentile (vote total). The target is `likes_percentile` in $(0, 1)$: within each calendar week, I compute the mean and standard deviation of $\log(1 + \text{likes})$, convert each post to a z-score within its week, then map the z-score through the standard normal CDF. This yields a comparable, stable scale across weeks and is invertible to log or raw likes using per-week statistics.

Auxiliary features. There are 18 numeric features: `day_of_week`, `hour_sin`, `hour_cos`, `text_length`, `num_words`, `num_hashtags`, `num_punctuation`, `is_question`, `caps_ratio`, `emoji_count`, `has_emoji`, `contains_number`, `time_since_last_post`, `week_index`, `weekly_post_count`, `log_weekly_post_count`, `week_log_like_mean`, and `week_log_like_std`. They are standardized with `StandardScaler` fit on the training set only.

Splitting. By default, the split is strictly temporal (70% train, 15% val, 15% test). In an attempt to improve model accuracy, I also added an optional feature to randomly split. I also used a “latest fraction” (e.g., 0.6, 0.75, 0.8, 1.0) for ablations, as the oldest 10% of posts existed over 4 years ago and may have less relevance to the posts of today. During attempted classification, I added an option to balance classes, so each class would contain the equal amount of posts during training.

Baseline. TF-IDF (1–2 grams, 10k features) on text only, then Ridge regression on the percentile target. I used `max_length=64` tokens instead of the default 128, as only 0.4% of posts exceed 64 BERT tokens (see `length.py`), so truncation loss is minimal while saving compute power.

BERT pipeline:

1. **DAPT:** Masked language modeling on posts and comments (domain adaptation).
2. **TAPT:** Masked LM on posts only, labels stripped (task adaptation).
3. **Supervised regression:** Load the adapted BERT, add a regression head and auxiliary tower.
 - A. Freeze BERT, train the MLP only. Important for incorporating auxiliary inputs and regression head.
 - B. Optionally unfreeze all or top- k layers with a low learning rate. Differential learning rates are used for the transformer vs. the head. I unfroze the top 4 layers during phase B testing.

Other models (comparison). Leveraged multiple model types with various tests on their hyperparameters and configuration. The first model was a Multi-Layer Perceptron on the MiniLM embeddings with appended auxiliary inputs. The second model was an MLP with a small transformer layer after embedding. I also did end-to-end DistilBERT (tokenize $\rightarrow$ [CLS]+aux tower $\rightarrow$ head). These are reported for comparison only.

Evaluation. I report RMSE and MAE on the percentile target, fraction of predictions within 0.5σ and 1.0σ , and MAE on raw likes (inverted from percentile). For classification, I report accuracy and F1.

4 Experiments

4.1 Dataset

Source. I use the SQLite database `yikyak_03_05.db`, which is a scraped collection of posts and comments of UCSD’s YikYak group from March 30, 2023 to March 5, 2026. The `posts` table has columns `text`, `created_at`, and `vote_total`. The `comments` table contains identical columns, although only the `text` column is used in the model for DAPT. Without filtering null text and posts with attachments, the database contains **51,342 posts** and **78,760 comments** in total. Of these, **46,796 posts** and **76,764 comments** are text-only. 4,546 posts and 1,996 comments have attachments. Supervised experiments use text-only posts (46,796). DAPT uses both posts and comments. A typical run uses the latest 80% of data with a temporal 70/15/15 split: Train 26,205, Val 5,615, Test 5,616.

	Posts	Comments
Total	51,342	78,760
Without attachments (text-only)	46,796	76,764
With attachments	4,546	1,996

Format. Text (string), datetime, signed integer vote count. Targets are derived: weekly-normalized percentile of $\log(1 + \text{likes})$ (primary regression target), or vote/percentile classes for classification.

Target choices. In an attempt to find the most accurate results, the pipeline supports multiple output styles. What began with simple log-vote regression changed into vote-based and percentile-based classification, then a single binary output, with and without sigmoid. The final regression target was decided to be a **weekly normalized percentile**. The target was found to be the most effective and is comparable across weeks for higher accuracy and variation accountability.

Splits. Strict temporal split by default; optional random split and “latest fraction” (e.g., 0.6, 0.75, 0.8, 1.0) for ablations.

4.2 Results

Main model comparison (percentile regression). Baseline from `baseline.py`; BERT pipeline best checkpoint: DAPT 3, TAPT 48, frozen 9, unfrozen 2, 0.8 ablation, temporal split (`dapt_e3_tapt_e48_frozen_e9_unfrozen_e2.ptrom`).

Model	RMSE	MAE	0.5σ	1.0σ	MAE raw
Baseline (TF-IDF + Ridge)	0.2952	0.2457	10.9%	21.8%	–
MLP (MiniLM + auxiliary)	0.2839	0.2411	0.3826	0.7252	16.21
Two Tower MLP	0.2859	0.2417	0.3775	0.7155	16.21
End2end DistilBERT (frozen)	0.2986	0.2375	42.65%	72.61%	71.48
BERT (DAPT 3, TAPT 90, frozen 4, unfrozen 1)	0.2807	0.2338	40.47%	73.40%	12.52
BERT (DAPT 3, TAPT 48, frozen 9, unfrozen 2)	0.2626	0.2191	42.17%	76.16%	12.15

TAPT epoch testing comparison. DAPT=3, latest 80% data, temporal split, frozen then unfrozen (top-4), early stopping:

TAPT epochs	Best val RMSE (frozen)	Best val RMSE (unfrozen)
20	0.2710	0.2701
48	0.2630	0.2643
90	0.2627	0.2637

4.3 Analysis

The BERT pipeline (DAPT 3, TAPT 48, frozen 9, unfrozen 2 with 0.8 ablation and a temporal split) achieves Test RMSE 0.2626 and MAE 0.2191, with 42.17% of predictions within 0.5σ and 76.16% within 1.0σ , and MAE on raw likes 12.15. It improves over the baseline (Test RMSE 0.2952, MAE 0.2457) by roughly 11% relative on RMSE and MAE. Domain and task adaptation (DAPT then TAPT) plus frozen/unfrozen regression yield a model that generalizes to chronologically later data (0.8 temporal split). With `max_length=64`, only 0.4% of posts are truncated. Classification results with 10 frozen epochs and 2 unfrozen epochs reach 46.14% accuracy, 0.3476 F1, and 0.4444 weighted F1.

5 Supplementary Material

A 5-minute video recording presenting motivation, approach, and visualization of results is required. Code is available in the project repository, and experiments can be reproduced via `run_all.py`.

References

- [1] Z. Cheng, J. Caverlee, and K. Lee, “Predicting twitter engagement with deep language models,” in *Companion Proceedings of the Web Conference 2020 (WWW)*. New York, NY, USA: ACM, 2020, pp. 399–404.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Minneapolis, MN, USA: Association for Computational Linguistics, 2019, pp. 4171–4186.
- [3] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. Online: Association for Computational Linguistics, 2020, pp. 8342–8357.